COMP4136
Natural Language Processing & Large Language Model

Lecture 8 Neural Language Models

Prof. MA Jing
Dept of Computer Science
Hong Kong Baptist University

香港浸會大學
HONG KONG BAPTIST UNIVERSITY

DEPARTMENT OF
COMPUTER SCIENCE
計算機科學系

Outline

- The Big Picture
- A fixed-window Neural Language Model
- RNN-based Language Model
- BERT
- GPT
- Roadmap: Pre-Training
- Roadmap: Fine-Tuning

Recap: Statistical Language Model

- Guess the next word...
  - ... I notice three guys standing on the ???
  - a="on the street"; b="on the computer" P(a)>P(b)
- Chain rule:
  - Given a sentence $\{w_1\ w_2\ \ldots w_n\}$

$$
\begin{aligned}
P(a) &= P(w_1 w_2 \ldots w_m) \\
&= P(w_1)P(w_2|w_1)P(w_3|w_1 w_2)\ldots P(w_m|w_1 w_2 \ldots w_{m-1}) \\
&= \prod_{k=1}^{m} P(w_k|w_1 \ldots w_{k-1})
\end{aligned}
$$

$$
\begin{aligned}
&P(\text{Do you still remember}) \\
&= P(\text{Do}) \times P(\text{you}|\text{Do}) \times P(\text{still}|\text{Do you}) \times P(\text{remember}|\text{Do you still})
\end{aligned}
$$

Recap: Why Language Model?

Many NLP tasks require **natural language output**:

- **Machine translation**: return text in the target language
- **Speech recognition**: return a transcript of what was spoken
- **Natural language generation**: return natural language text
- **Spell-checking**: return corrected spelling of input
- …

Language models define **probability distributions over (natural language) strings or sentences**.

➔ We can use a language model to **score possible output strings** so that we can choose the best (i.e. most likely) one:

if $P_{LM}(A) > P_{LM}(B)$, return A, not B

Problems with N-gram LMs

Sparsity Problems

e.g., Predict: "the students opened their ???"

Sparsity Problem 1

Problem: What if "*students opened their w*" never occurred in data? Then *w* has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count(students opened their } \boldsymbol{w})}{\text{count(students opened their)}}$$

Sparsity Problem 2

Problem: What if "*students opened their*" never occurred in data? Then we can't calculate probability for *any w*!

(Partial) Solution: Just condition on "*opened their*" instead. This is called *backoff*.

Note: Increasing n makes sparsity problems worse. Typically, n< 5.

DEPARTMENT OF COMPUTER SCIENCE 計算機科學系
香港浸會大學 HONG KONG BAPTIST UNIVERSITY

Problems with N-gram LMs

Storage Problems

e.g., Predict: "the students opened their ???"

Storage: Need to store count for all *n*-grams you saw in the corpus.

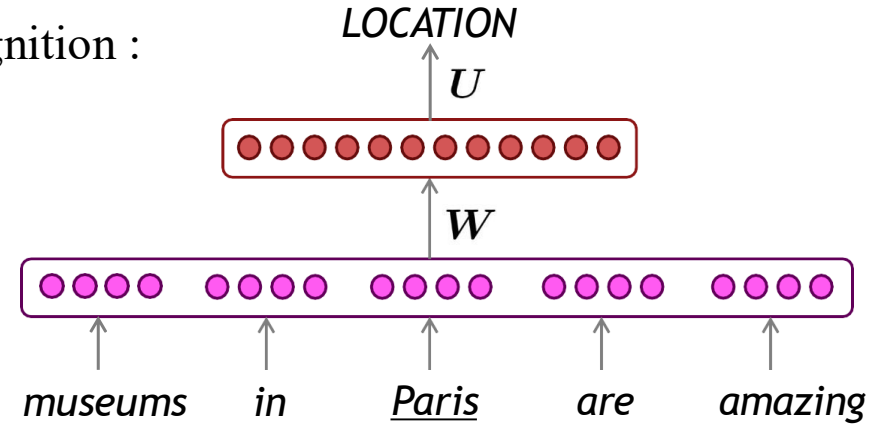
$$P(\boldsymbol{w}|\text{students opened their}) = \frac{\text{count}(\text{students opened their } \boldsymbol{w})}{\text{count}(\text{students opened their})}$$

Increasing *n* or increasing corpus increases model size!

Neural Language Model

- Recall the Language Modeling task:
  - Input: sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - Output: prob. dist. of the next word $P(\boldsymbol{x}^{(t+1)} | \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$

- We are looking for f_θ, so that
  $$P(x_i|x_1, \ldots, x_{i-1}) = f_\theta(x_i|x_1, \ldots, x_{i-1})$$
- Can f_θ be a window-based neural model?
  - E.g., Named Entity Recognition :



A fixed-window neural Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

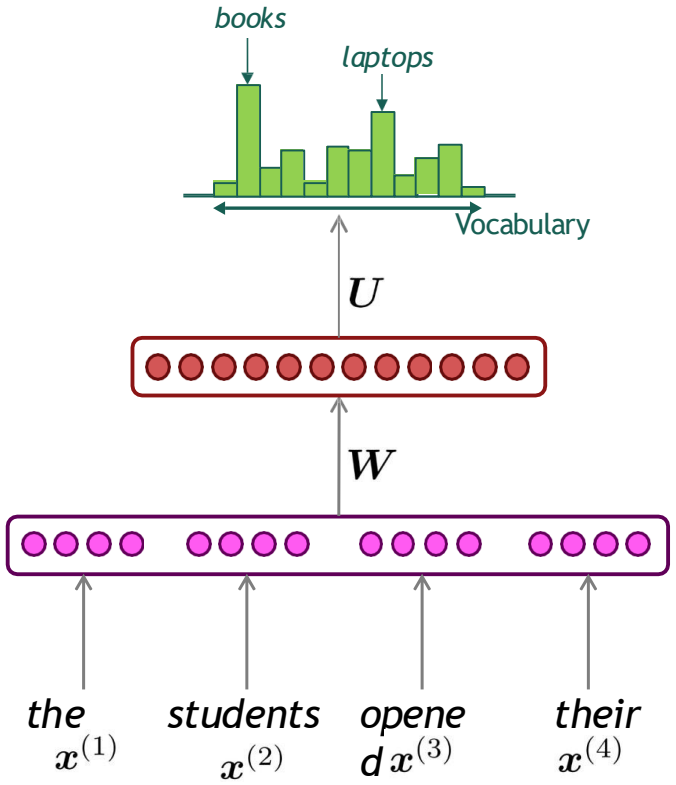
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



A fixed-window neural Language Model

Compared with *n*-gram LM:

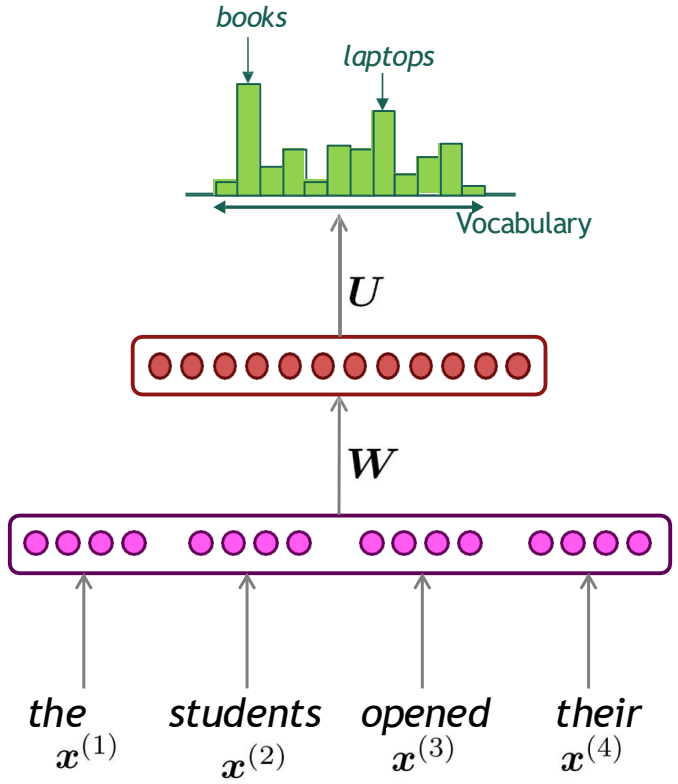
- No sparsity problem
- Don't need to store all observed *n*-grams

Problems:

- Fixed window is too small
- Enlarging window enlarges *W*
- Window can never be large enough!

We need a neural architecture that can process *any length input*

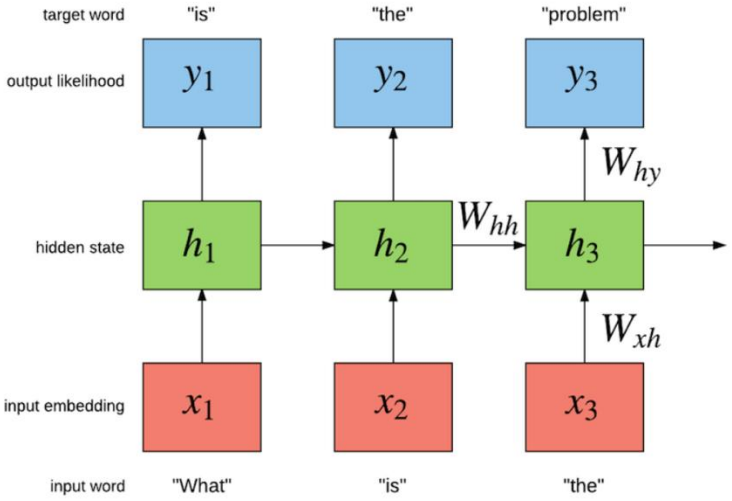
***Recurrent NN** is the solution !*



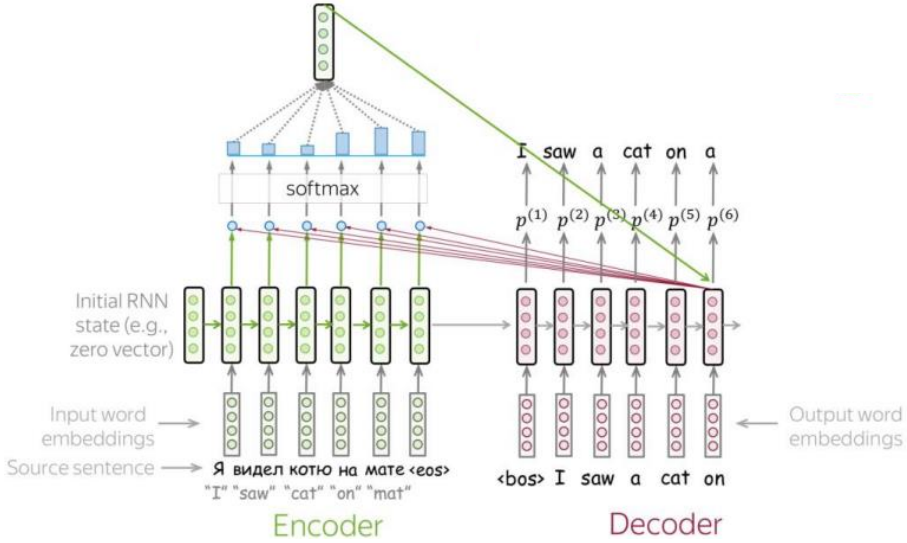
Ref: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

RNN-based Language Model

- Text Generation.



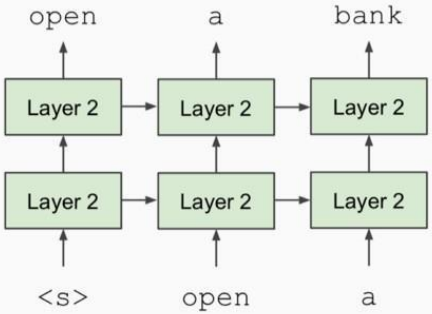
- Seq2Seq.



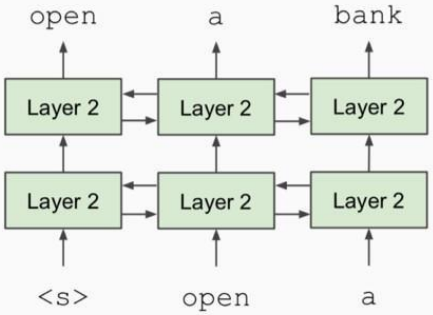
Bidirectional Encoders

- Some Language models only use left context or right context (although ELMo used two independent LMs from each direction).
- Language understanding is bidirectional

Unidirectional context
Build representation incrementally



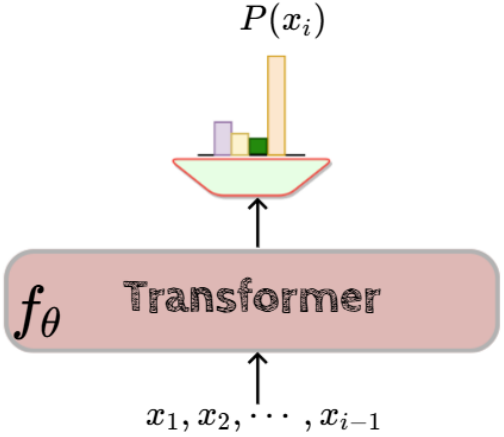
Bidirectional context
Words can "see themselves"



Neural Language Model

- Recall the Language Modeling task:
  - Input: sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - Output: prob. dist. of the next word $P(\boldsymbol{x}^{(t+1)} | \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)})$

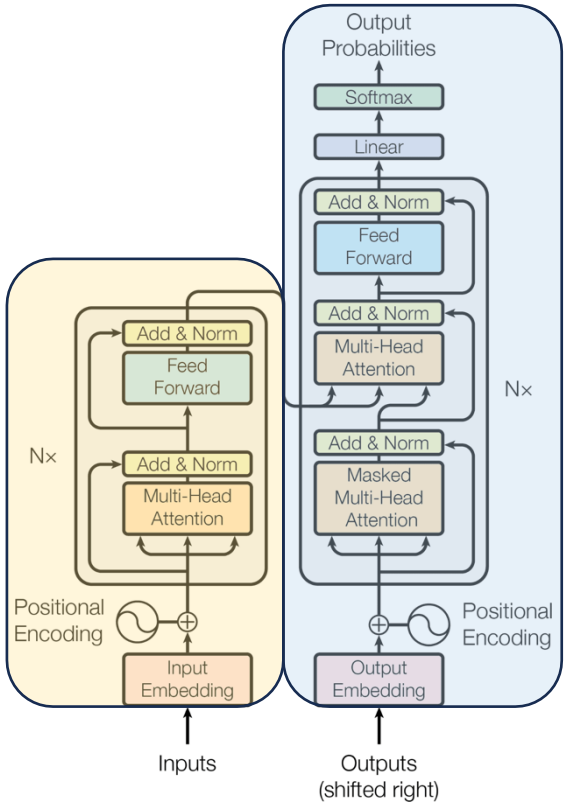
- We are looking for f_θ, so that
  $$P(x_i|x_1, \ldots, x_{i-1}) = f_\theta(x_i|x_1, \ldots, x_{i-1})$$
- Can f_θ be a **Transformer**?



Recap: Transformers, 2017

Input – input tokens
Output – hidden states

Representation



Input – output tokens and hidden states*
Output – output tokens

Generation

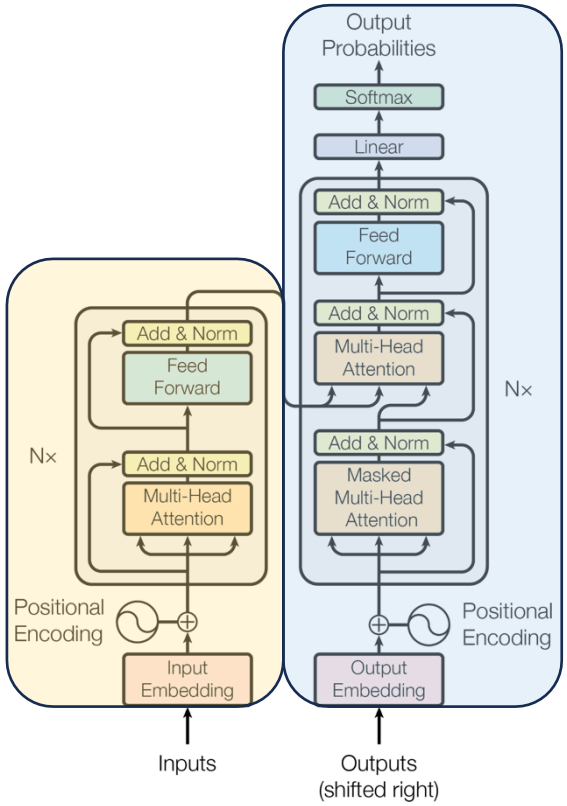
Recap: Transformers, 2017

Input – input tokens
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Representation



Input – output tokens and hidden states*
Output – output tokens

Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

Generation

Recap: Transformers, 2017

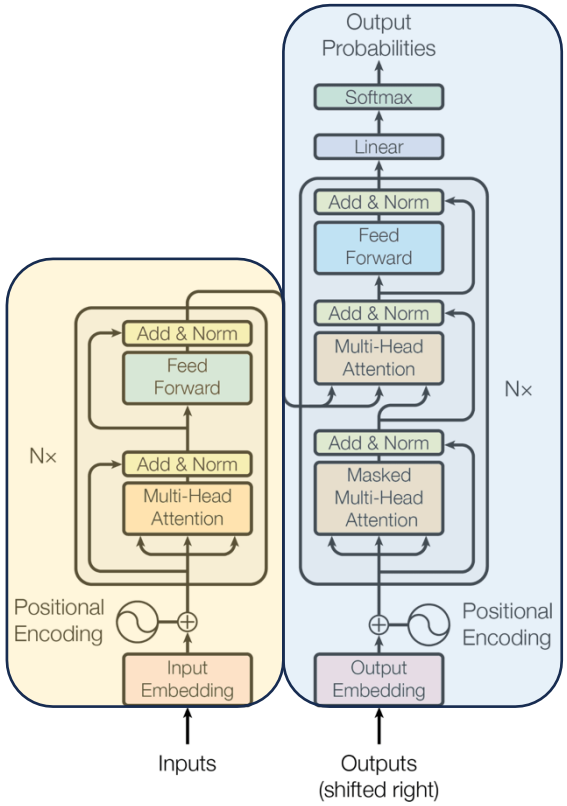
Input – input tokens
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Can also be adapted to generate tokens *by appending a module that maps hidden state dimensionality to vocab size*

Representation



Input – output tokens and hidden states*
Output – output tokens

Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

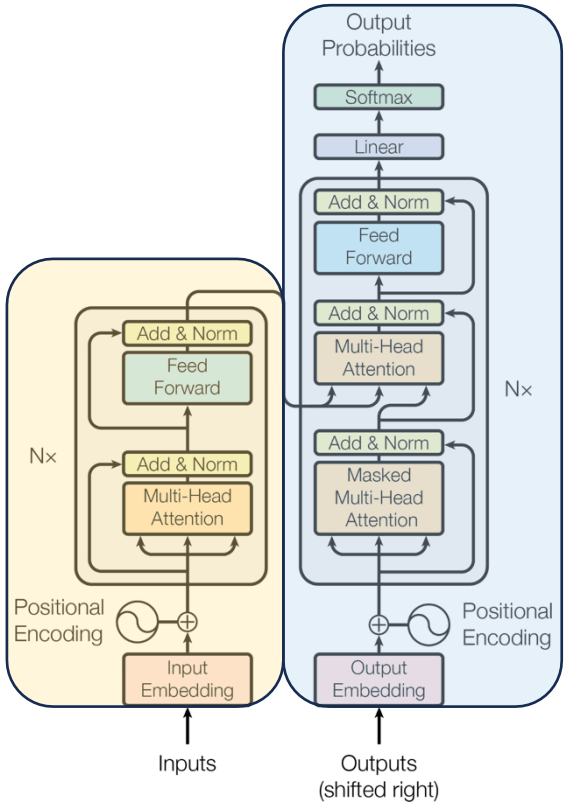
Can also be adapted to generate hidden states *by looking before token outputs*

Generation

Recap: Transformers, 2017

BERT
Oct 2018

Representation



GPT
Jun 2018

Generation

BERT-Bidirectional Encoder Representations

BERT Pre-Training Corpus:

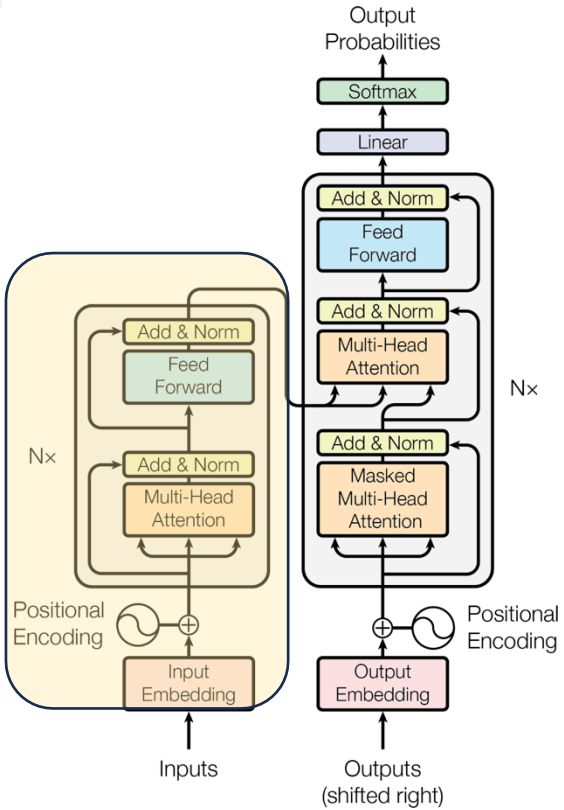
- English Wikipedia - 2,500 million words
- Book Corpus - 800 million words

BERT Pre-Training Tasks:

- MLM (Masked Language Modeling)
- NSP (Next Sentence Prediction)

BERT Pre-Training Results:

- BERT-Base – 110M Params
- BERT-Large – 340M Params



Ref: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, NAACL 2019

BERT: Task1

MLM (Masked Language Modeling)

i ___ $to\ read\ a$ ___ $[eos]$

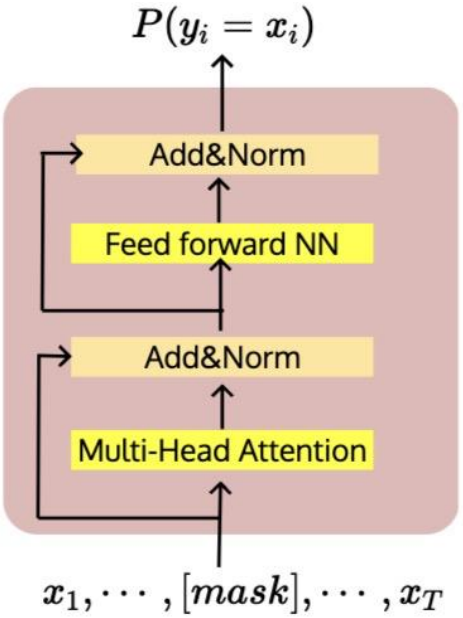


Transformer Decoder? **No**



Transformer Encoder

$$P(y_i|x_1, \ldots, x_i = [mask], \ldots, x_T) = ?$$



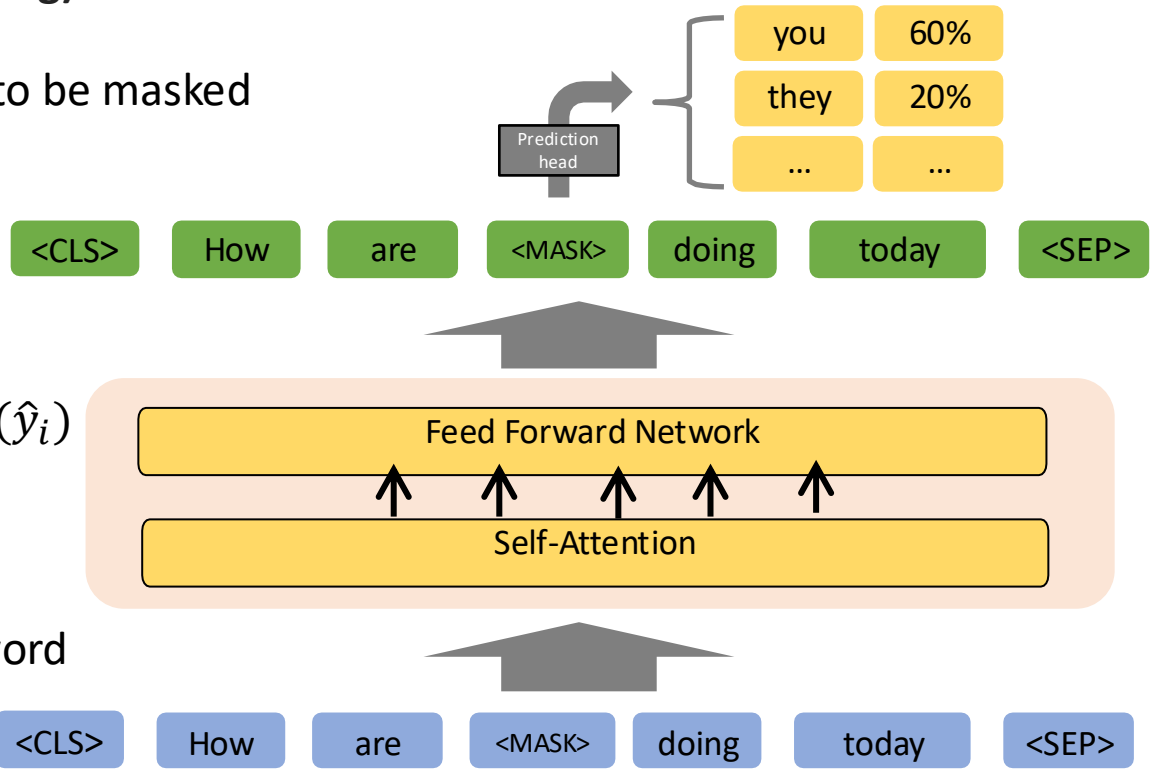
BERT: Task1

MLM (Masked Language Modeling)

use [mask] token for the words to be masked

$$[\text{mask}] \in V$$

$$h_{[mask]}?$$



Learning Objective

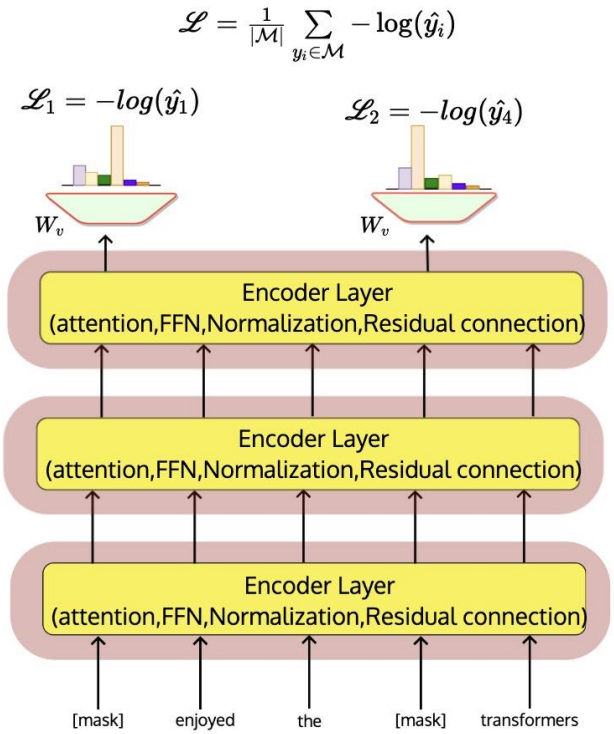
$$Loss = \frac{1}{|M|} \sum_{y_i \in M} -\log(\hat{y}_i)$$

M: a set of masked words

$y_i, \hat{y}_i$: ground-truth/predicted word

Note: 15% of words are masked

BERT: Task1



BERT Base model: 12 layers with 12 Attention heads/per layer

15% masked words are sampled uniformly

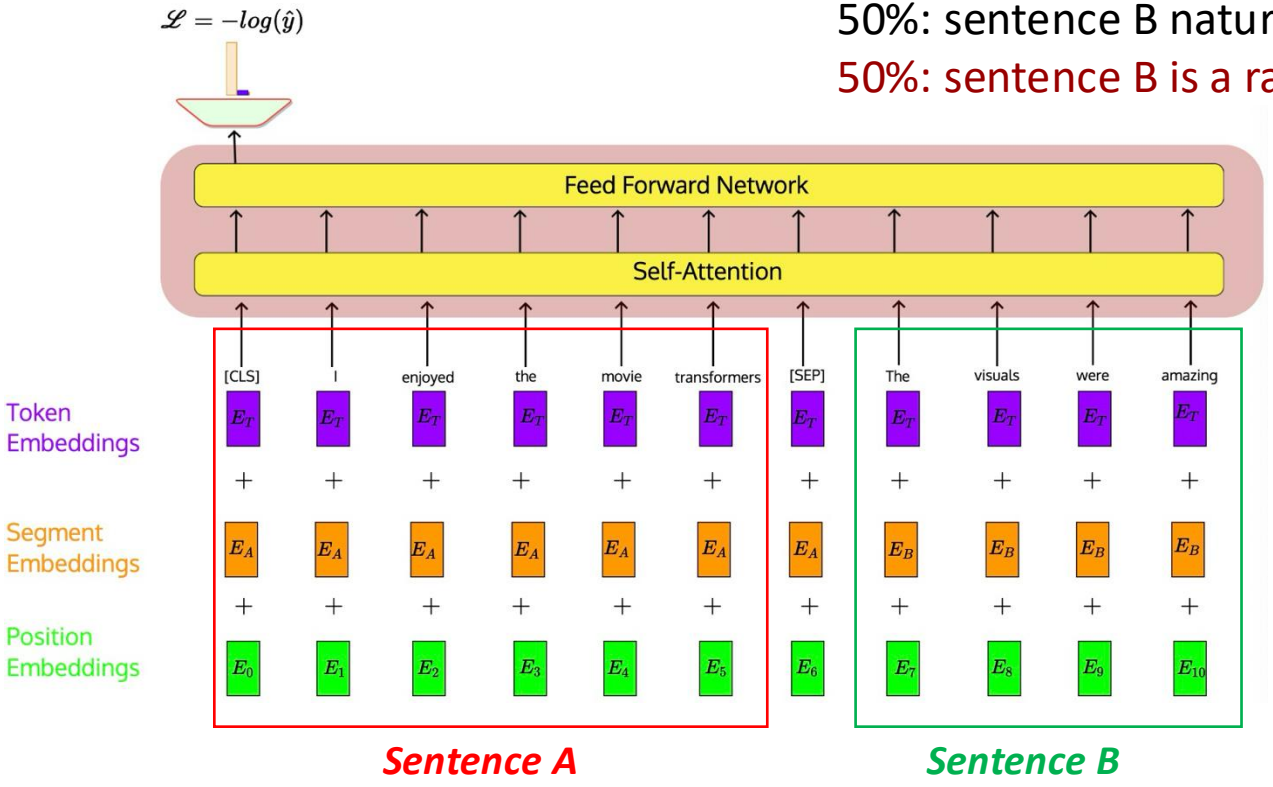
BERT: Task2

Next Sentence Prediction (NSP)

Training:

50%: sentence B naturally follows sentence A

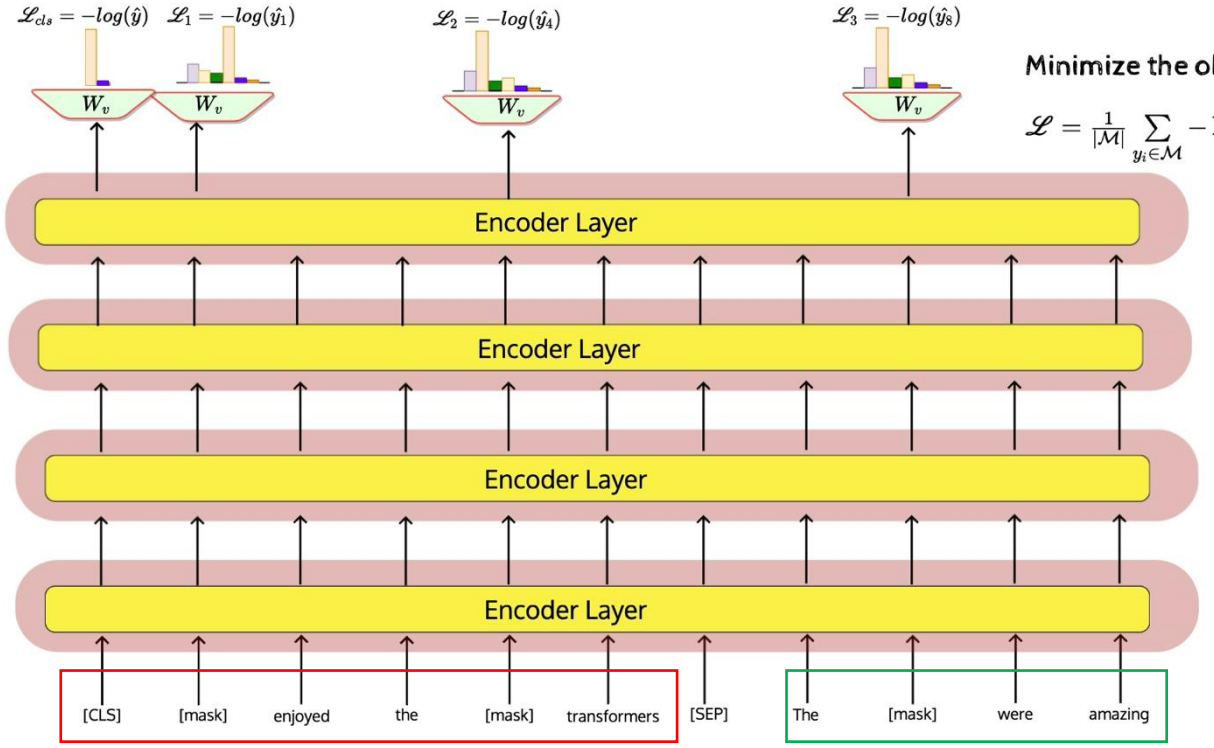
50%: sentence B is a random sentence



a separate learnable segment embedding is used in addition to token and positional embeddings

BERT: Task2

Next Sentence Prediction (NSP)



Training

Minimize the objective:

$$\mathcal{L} = \frac{1}{|\mathcal{M}|} \sum_{y_i \in \mathcal{M}} -\log(\hat{y}_i) + \mathcal{L}_{cls}$$

a classification loss: whether **B** is the next sentence of A?

BERT: Pre-Training

Pre-training & Model Size

Dataset:



BookCorpus

800M words

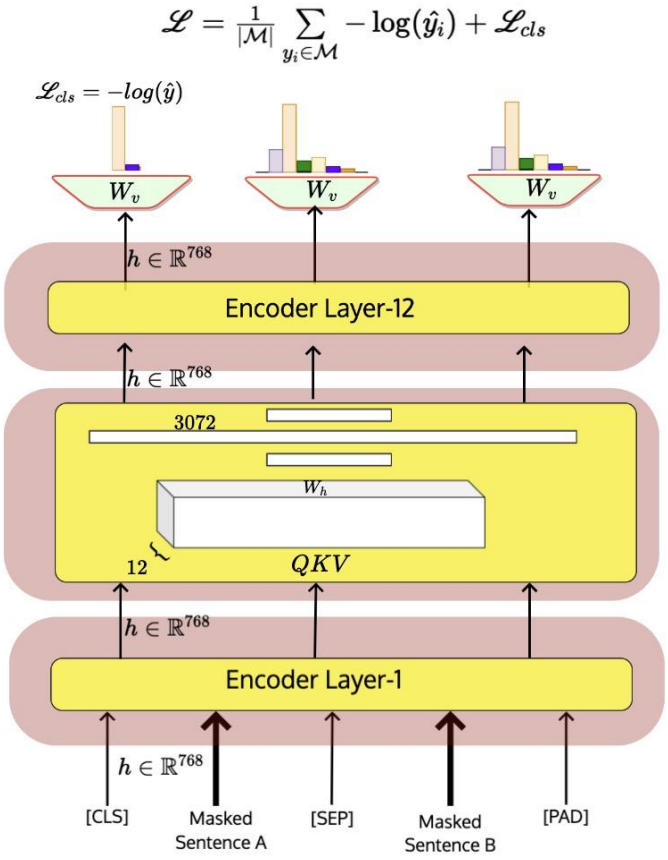


WikiPedia

2500M words

Vocabulary size: 30,000
Context length: ≤512 tokens

Model Architecture:
Num. of layers:12 (base)/24(large)
Hidden size: 768/1024
Num. of Attention heads:12 /16
Intermediate size: 4H FFN= 3072 /4096



BERT: Pre-Training

Pre-training & Model Size

Embedding layer:

Token Embeddings

$30000 \times 768 \approx 23M$

Segment Embeddings

$2 \times 768 \approx 1536$

Position Embeddings

$512 \times 768 \approx 0.4M$

Total: 23.4 M

For a Single Layer:

Self-Attention (Ignoring bias)

W_K, W_Q, W_V, A

$(768 \times 64 \times 3) \times 12 = 1.7M$

$768 \times 768 = 0.6M$

FFN

$768 \times 3072 + 3072 \times 768 + (3072 + 768) = 4.7M$

Total: ≈ 7M

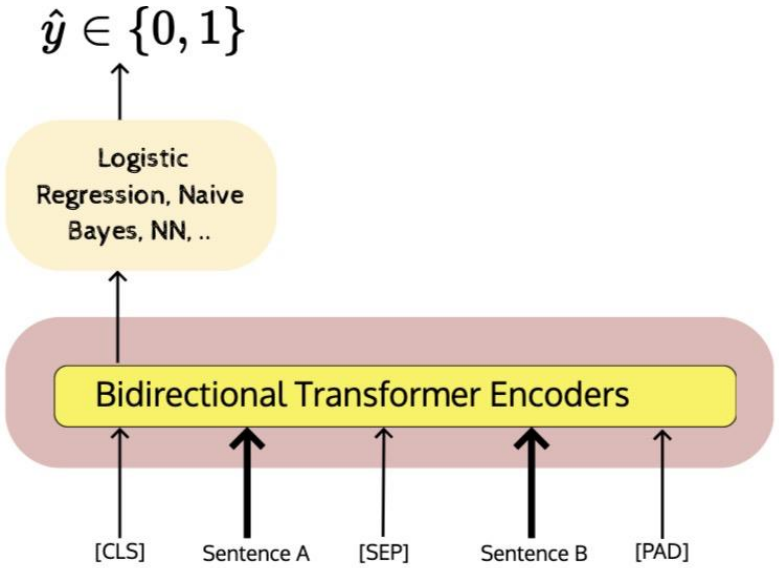
For 12 layers: ≈ 84M

Total number of Parameters: 23.4+84 = 107.4M(110M in Paper)

*Actual vocabulary size 30522, parameters for layer normalization are excluded

BERT: Applications

BERT as feature extractor



Input:
a sentence of length less than 512

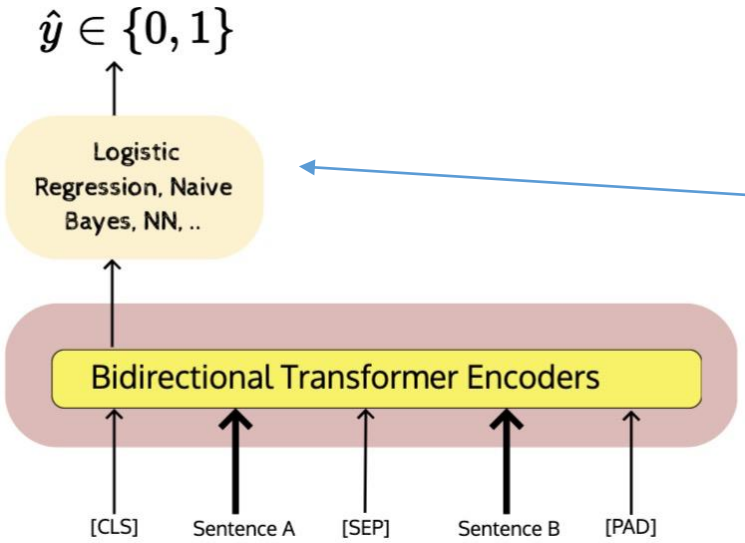
Feature vector:
the final hidden representation (output of the last encoder)

Classification:
use any ML model

All the parameters of BERT are freezed and only the classification head is trained from scratch.

BERT: Applications

Fine-tuning BERT for classification task



Input:
a sentence of length less than 512

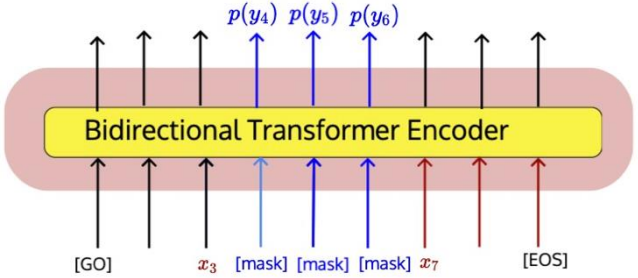
Classification head:
any suitable ML model
Initialize the parameters of the classification head randomly.

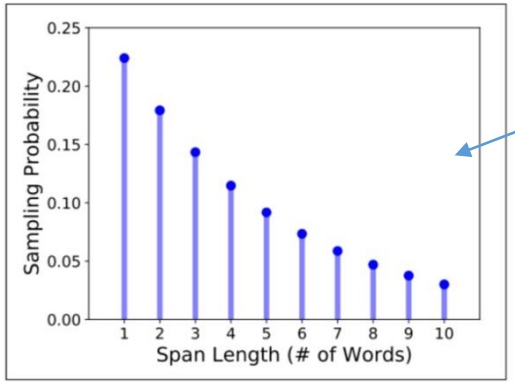
Model Training:
train the entire model including the parameters of the pre-trained BERT

do not mask words
the model converges quickly

BERT: Variations

SpanBERT





BERT:
masking tokens randomly

Mask a contiguous span of random tokens: natural for tasks like QA

Span length:
random variable that follows the Geometric Distribution

Ref: SpanBERT: Improving Pre-training by Representing and Predicting Spans

BERT: Variations

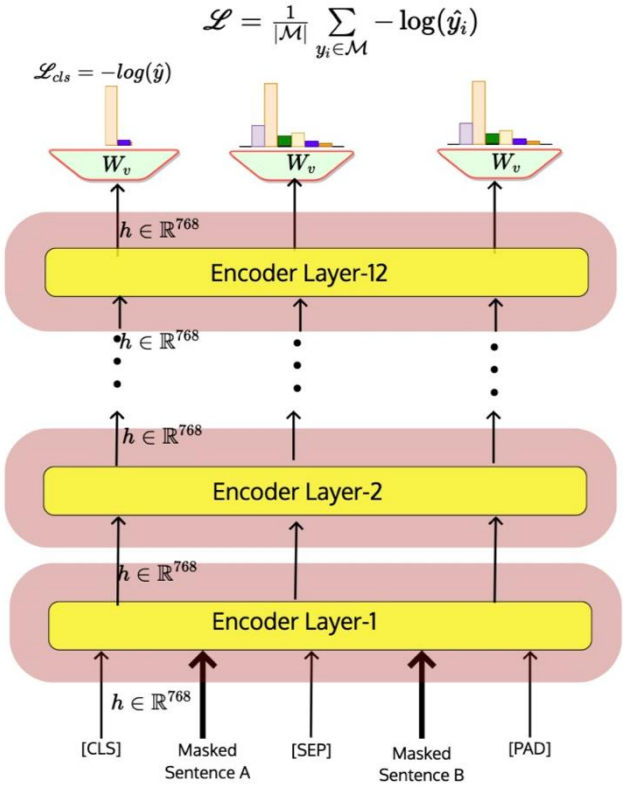
RoBERTa

BERT:

training is an expensive process

Proposed a set of Robust design choices for BERT

- Training the model longer, with bigger batches (10 to 30 fold) , over more data (almost 10-fold).
- remove the next sentence prediction objective
- training on longer sequences
- dynamically changing the masking pattern applied to the training data



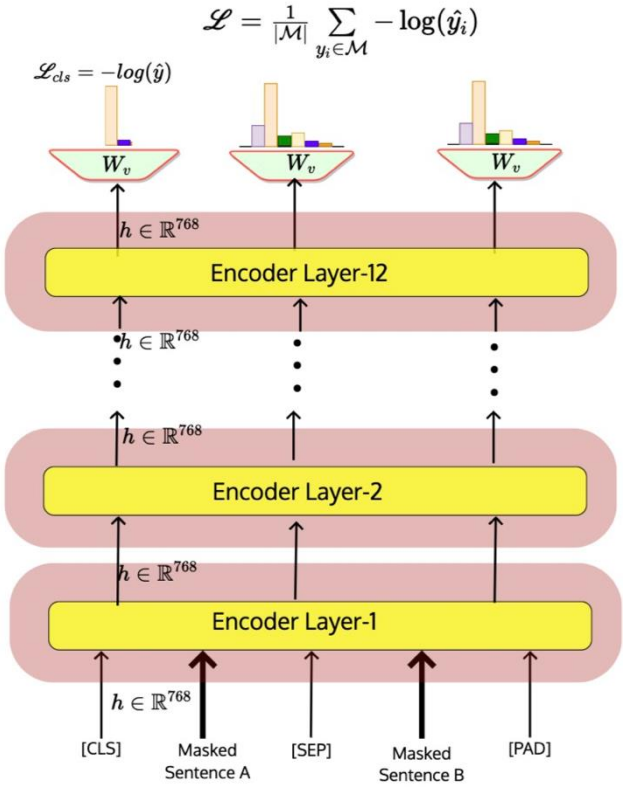
Ref: RoBERTa: A Robustly Optimized BERT Pretraining Approach

BERT: Variations

ALBERT-TinyBERT-DistillBERT

To reduce the number of parameters in BERT using *factorization of embedding matrix*, *weight sharing* and *knowledge distillation (KD) techniques*.

Model size
108MB → 12 MB
5 to 9 times speedup in inference time



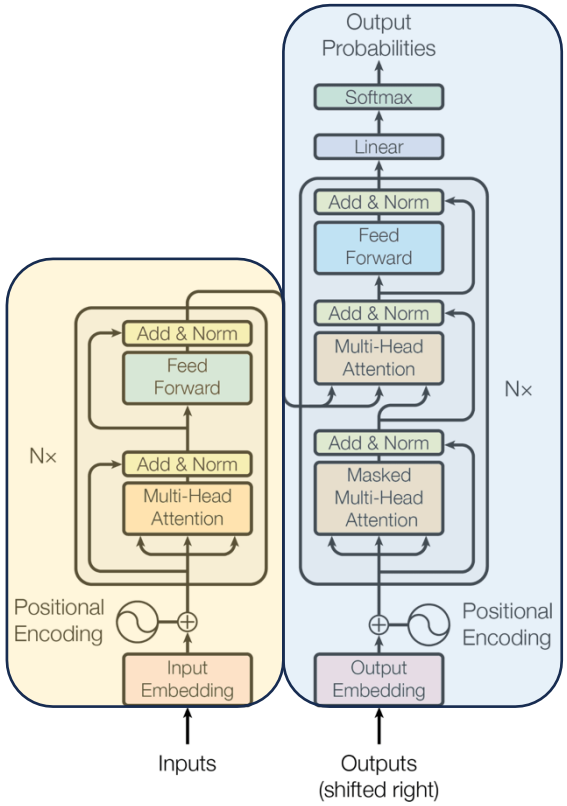
Ref:

- ALBERT: A LITE BERT FOR SELF-SUPERVISED LEARNING OF LANGUAGE REPRESENTATIONS
- TinyBERT: Distilling BERT for Natural Language Understanding
- DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter

Recap: Transformers, 2017

BERT
Oct 2018

Representation



GPT
Jun 2018

Generation

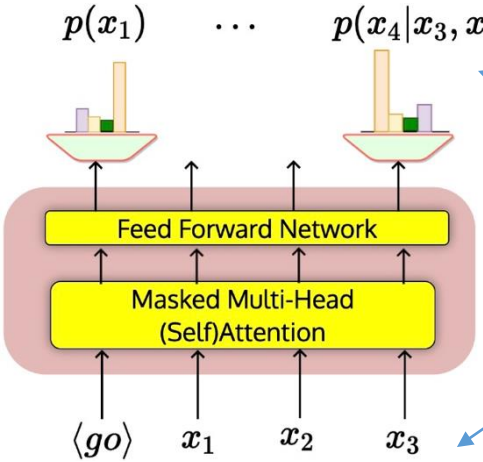
GPT - Generative Pretrained Transformer

GPT Pre-Training Corpus:

- Similarly, BooksCorpus and English Wikipedia

GPT Pre-Training Tasks:

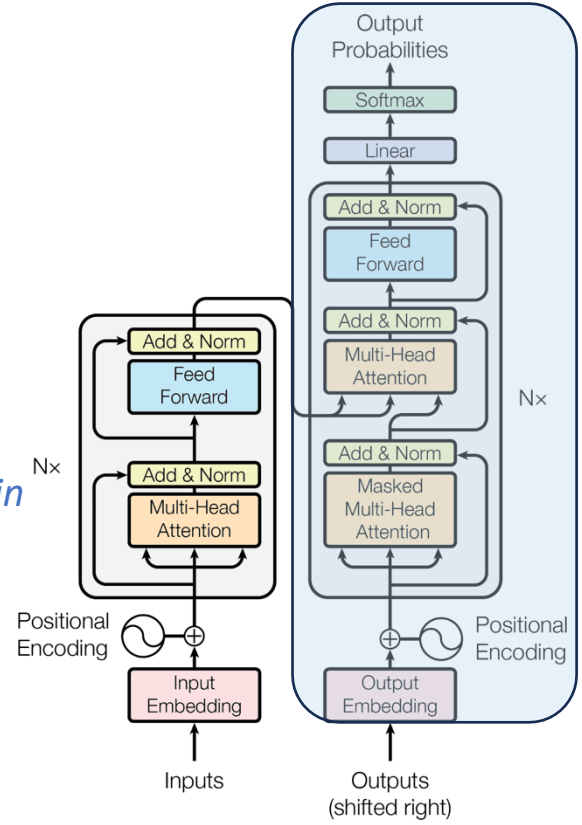
- Predict the next token, given the previous tokens



Outputs: each term in the chain rule

No cross attention layers

Inputs: word sequence



GPT

create a stack (n) of the modified decoder layers (also called as transformer block)

X: input word sequence

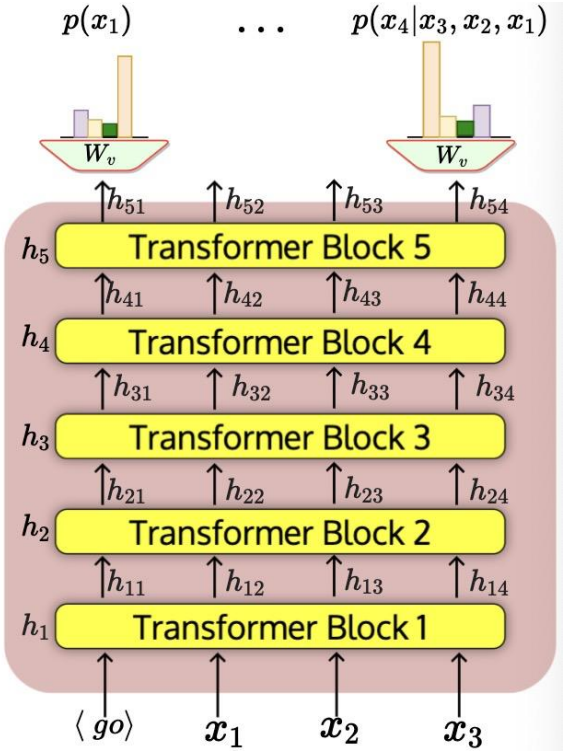
$h_0 = X$

$h_l = trans_block(h_{l-1}), l \in [1, n]$

Output

$$P(x_i) = softmax(h_n[i]W_v)$$

$$\mathscr{L} = \sum_{i=1}^{T} \log(P(x_i|x_1, \cdots, x_{i-1}))$$

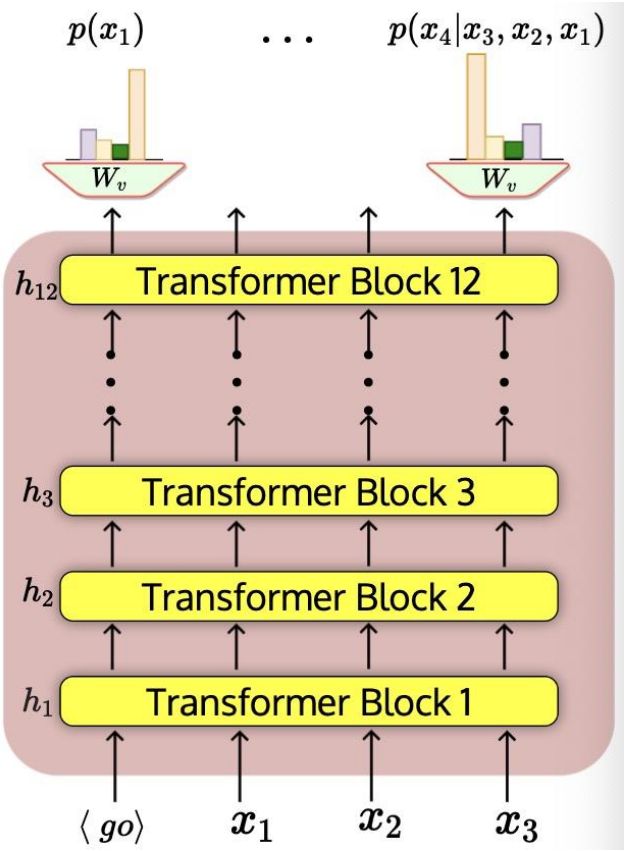


GPT

Input data:
BookCorpus: 7000 unique books, 74 Million sentences and approximately 1 Billion words
Vocab size |V|: 40478
Embedding dim: 768

Model
Contains 12 decoder layers (transformer blocks)
Context size: 512
Attention heads: 12
FFN hidden layer size: 768×4=3072

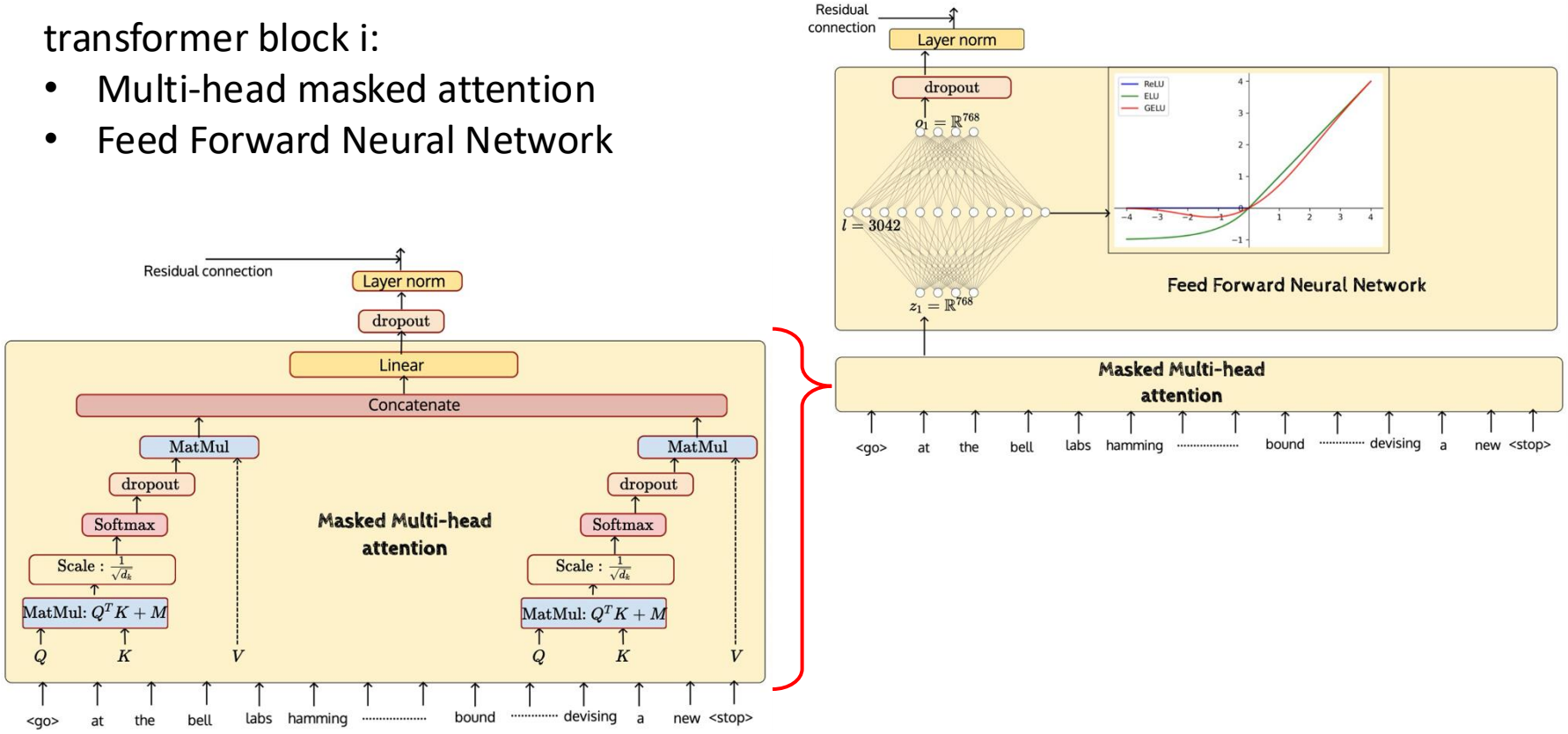
Dropout, layer normalization, and residual connections etc



GPT

transformer block i:

- Multi-head masked attention
- Feed Forward Neural Network



GPT

Number of parameters

Token Embeddings:

$$40478 \times 768 = 31 \times 10^6 = 31M$$

Position Embeddings

$$512 \times 768 = 0.3 \times 10^6 = 0.3M$$

Total: 31.3 M

Layer	Parameters (in Millions)
Embedding Layer	31.3
Attention layers	27.6
FFN Layers	56.4
Total	116.461056*

Attention parameters per block

$$W_K = W_Q = W_V = 768 \times 64$$

12 attention head:

$$12 \times 3 \times (768 \times 64) \approx 1.7M$$

For a Linear layer:

$$768 \times 768 \approx 0.6M$$

For all 12 blocks

$$12 \times 2.3 = 27.6M$$

FFN parameters per block

$$2 \times (768 \times 3072) + 3072 + 768 = 4.7 \times 10^6 = 4.7M$$

For 12 blocks: $12 \times 4.7 = 56.4M$

Common roadmap for LLMs

- Phase 1: pre-training
  - Learn **general** world knowledge, ability, etc.
- Phase 2: Supervised finetuning
  - Tailor to **tasks** (**unlock** some abilities)
- Phase 3: RLHF
  - Tailor to **humans**
  - *Even you could teach ChatGPT to do something*

Pre-Training

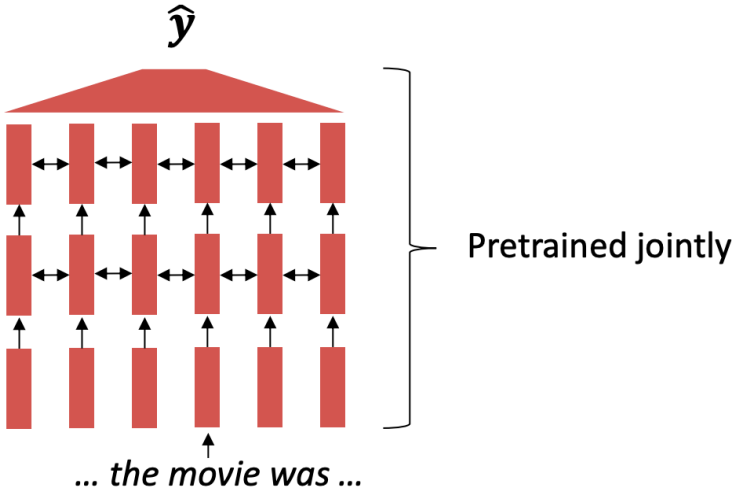
Why Pretraining?

In modern NLP:

- All (or almost all) parameters in NLP networks are initialized via **pretrianing.**
- Pretraining methods hide parts of the input from the model, and then train the model to reconstruct those parts.

This has been exceptionally effective at building strong:

- **representations of language**
- **parameter initializations** for strong NLP models.
- **probability distributions** over language that we can sample from



[This model has learned how to represent entire sentences through pretraining]

Pre-Training

Self-supervised training algorithm

We just train them to predict the next word!

1. Take a corpus of text
2. At each time step *t*
   i. ask the model to predict the next word
   ii. train the model using gradient descent to minimize the error in this prediction

"**Self-supervised**" because it just uses the next word as the label!

Pre-Training

Training loss

Cross-Entropy (CE) Loss: The negative log probability that the model assigns to the true next word *w*

- Update the model weights in the direction that assigns a higher probability to *w*;

CE loss: difference between the correct probability distribution and the predicted distribution

$$L_{CE} = -\sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w]$$

- The correct distribution $\mathbf{y}_t$ knows the next word, so is 1 for the actual next word and 0 for the others.
- So in this sum, all terms get multiplied by zero except one: the logp the model assigns to the correct next word, so:

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

Pre-Training

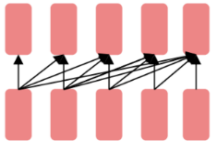
Teacher forcing

- At each token position *t*, model sees correct tokens $w_{1:t}$,
  - Computes loss (–log probability) for the next token w_{t+1}
- At next token position t+1 we ignore what model predicted for w_{t+1}
  - Instead we take the **correct** word w_{t+1}, add it to context, move on

Pre-Training

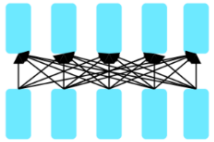
Pretraining for three types of architectures

The neural architecture influences the type of pretraining, and natural use cases.



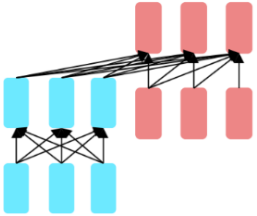
Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words
- **Examples:** GPT-2, GPT-3, LaMDA



Encoders

- Gets bidirectional context – can condition on future!
- Wait, how do we pretrain them?
- **Examples:** BERT and its many variants, e.g. RoBERTa



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?
- **Examples:** Transformer, T5, Meena

Pretrained Encoders

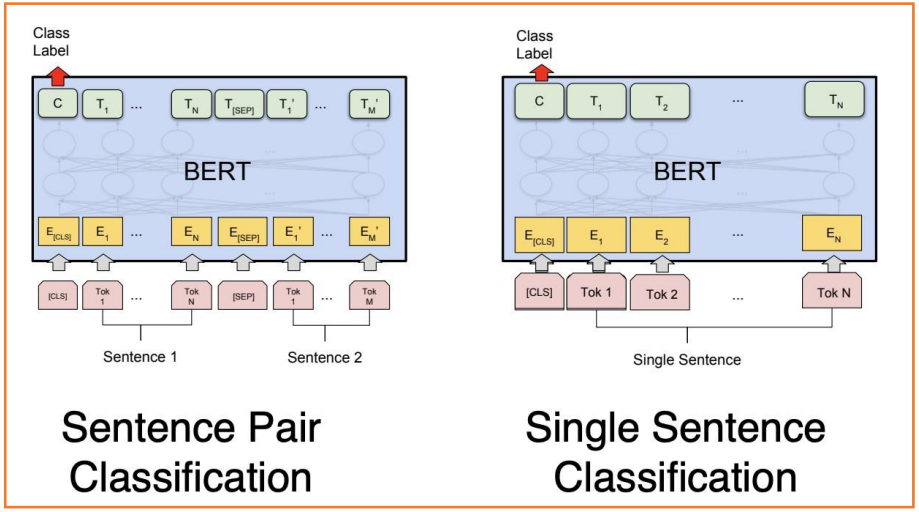
BERT

- **Fully bidirectional transformer encoder**
  - BERTbase: 12 layers, hidden size=768, 12 att'n heads (110M parameters)
  - BERTlarge: 24 layers, hidden size=1024, 16 att'n heads (340M parameters)
- **Input:** sum of token, positional, segment embeddings
  - **Segment embeddings** (A and B): is this token part of sentence A (before SEP) or sentence B (after SEP)?
- **[CLS]** and **[SEP]** tokens: added during pre-training
- **Pre-training tasks:**
  - Masked language modeling
  - Next sentence prediction

Pretrained Encoders

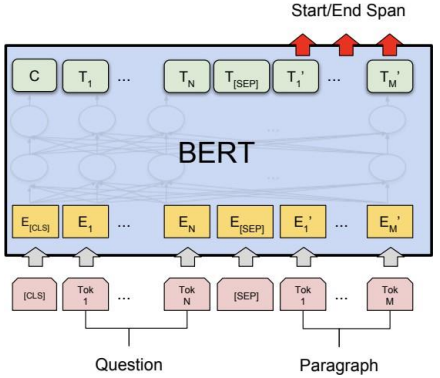
Using BERT for Classification

Add a **softmax classifier** on final layer of **[CLS]** token



Using BERT for Question-Answering

Input: [CLS] question [SEP] answer passage [SEP]
Learn to predict a START and an END token on answer tokens

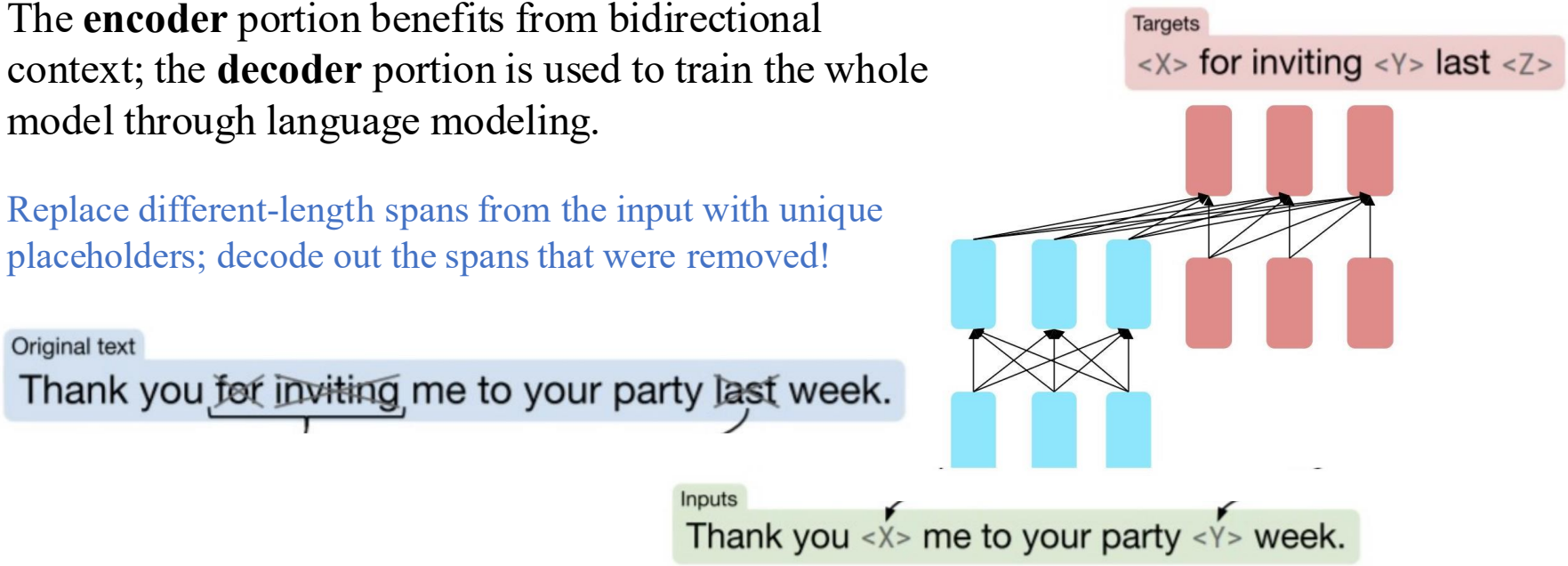


Pretrained Encoder-Decoders

T5

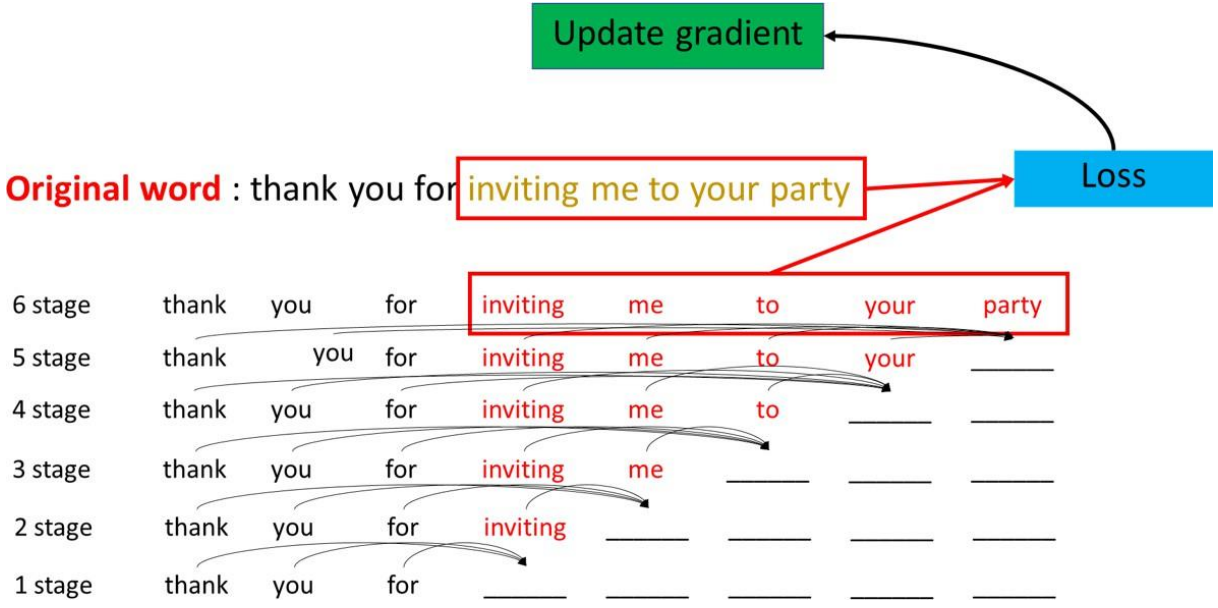
The **encoder** portion benefits from bidirectional context; the **decoder** portion is used to train the whole model through language modeling.

Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!



Pretrained Decoders

It's natural to pretrain decoders as language models and then use them as generators, finetuning their $p_\theta(w_t|w_{1:t-1})$!



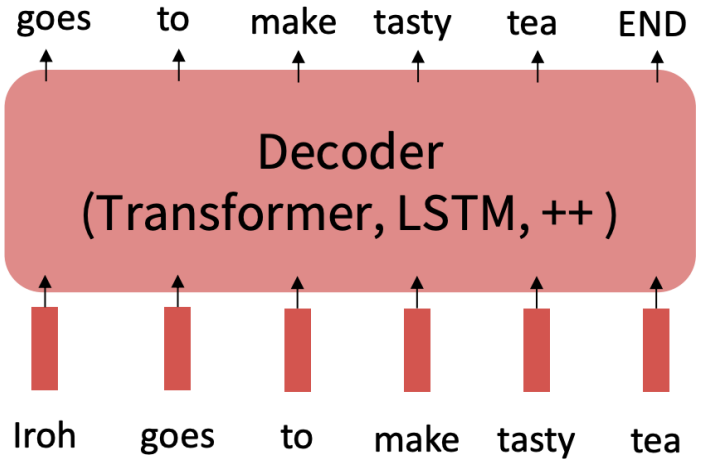
Pretrained Decoders

Recall the language modeling task:

- Model the probability distribution over words given their past contexts.
- There's lots of data for this! (In English.)

Pretraining through language modeling:

- Train a neural network to perform language modeling on a large amount of text.
- Save the network parameters.



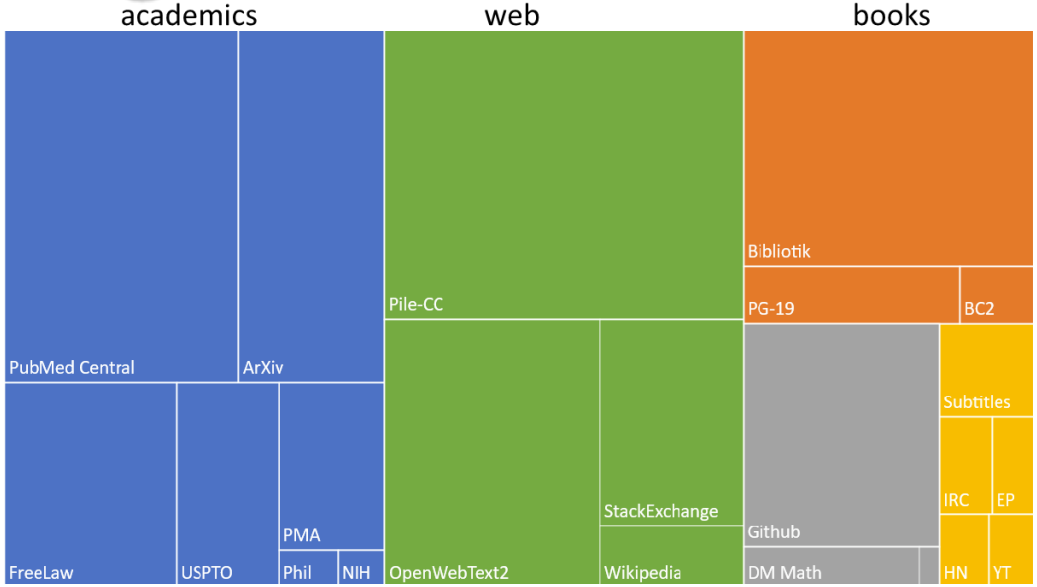
Pretraining data for LLMs

LLMs are mainly trained on the web

Filtering for quality and safety

Quality is subjective

- Many LLMs attempt to match Wikipedia, books, particular websites
- Need to remove boilerplate, adult content
- Deduplication at many levels (URLs, documents, even lines)



Safety also subjective

- Toxicity detection is important, although that has mixed results
- Can mistakenly flag data written in dialects like African American English

Common roadmap for LLMs

- Phase 1: pre-training
  - Learn **general** world knowledge, ability, etc.
- Phase 2: Supervised finetuning
  - Tailor to **tasks** (**unlock** some abilities)
- Phase 3: RLHF
  - Tailor to **humans**
  - *Even you could teach ChatGPT to do something*

Text contains enormous amounts of knowledge
Pretraining on lots of text with all that knowledge is what gives language models their ability to do so much

Finetuning

Why Finetuning?

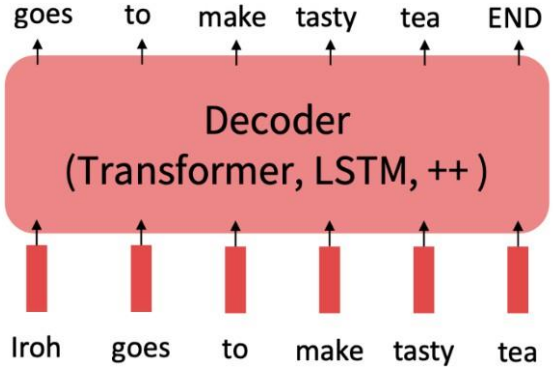
- if we need our LLM to work well on a domain it didn't see in pretraining.

Pretraining VS Finetuning Paradigm

Pretraining can improve NLP applications by serving as parameter initialization.

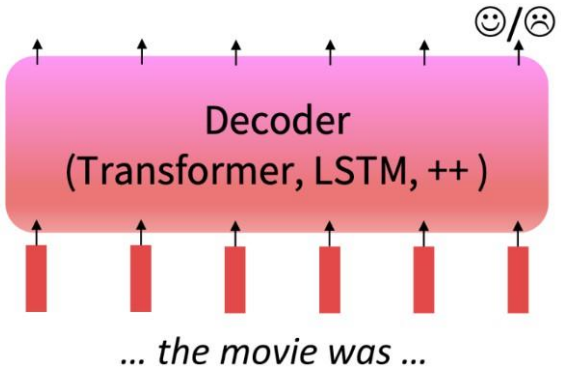
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



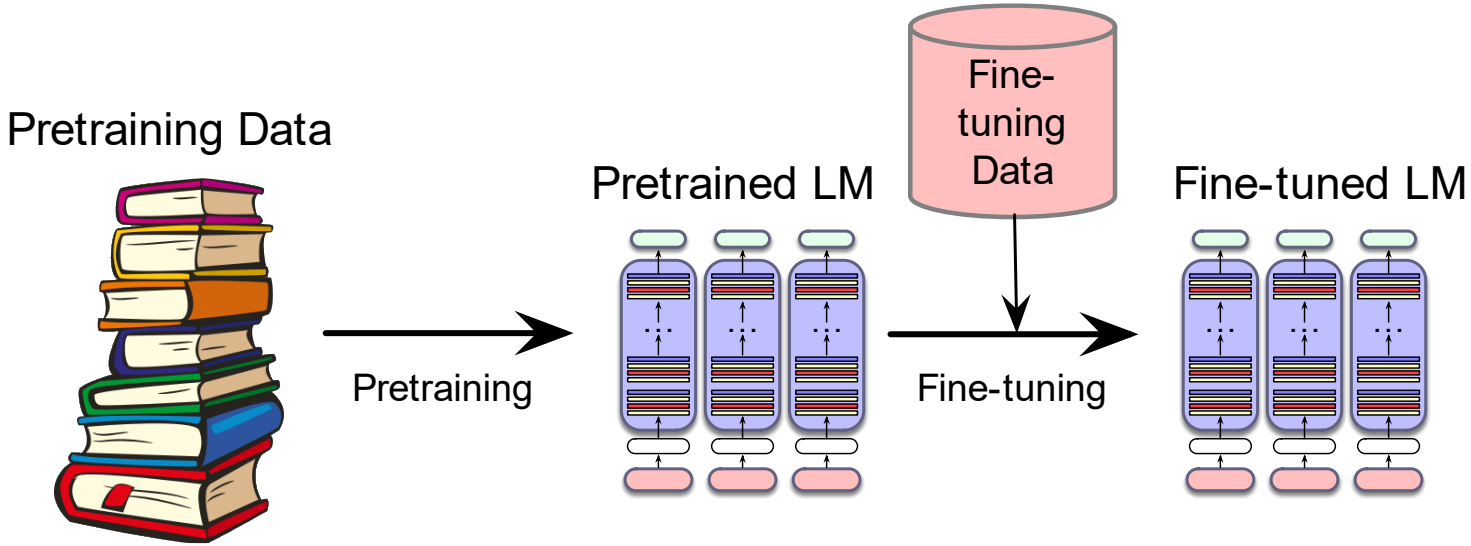
Step 2: Finetune (on your task)

Not many labels; adapt to the task!



Finetuning

- taking a pretrained model and further adapting some or all of its parameters to some new data
- Fine-tuning involves adapting model for various downstream tasks (with a minimal change in the architecture)



SFT - Supervised Fine-Tuning

Labeled Dataset: $\{x_1, x_2, \ldots, x_m\}, y$

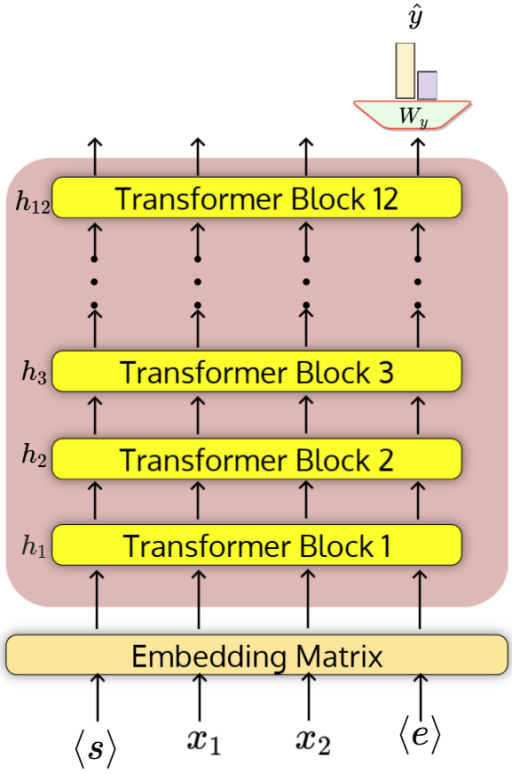
Input: add additional tokens based on the type of downstream task. E.g., start ⟨s⟩ and end ⟨e⟩ tokens for classification tasks

Output: replace the pre-training LM head with the classification head (e.g., a linear layer W_y)

$$\hat{y} = P(y|x_1, \cdots, x_m) = softmax(W_y h_l^m)$$

Training Loss:

$$\mathscr{L} = -\sum_{(x,y)} \log(\hat{y}_i)$$



Also called instruction fine-tuning (SFT)

SFT - Example

Sentiment Analysis

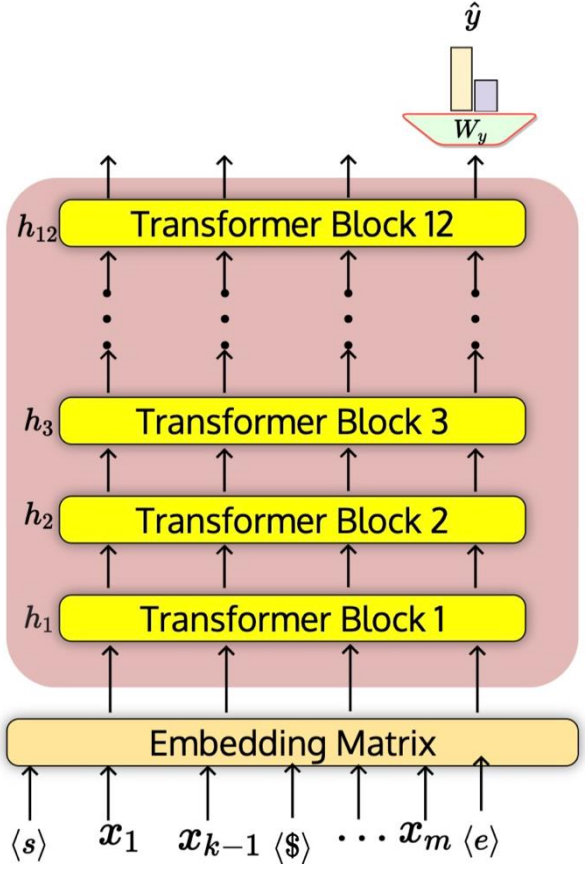
Text: Wow, India has now reached the moon
Sentiment: **Positive**

$$\hat{y} \in \{+1, -1\}$$

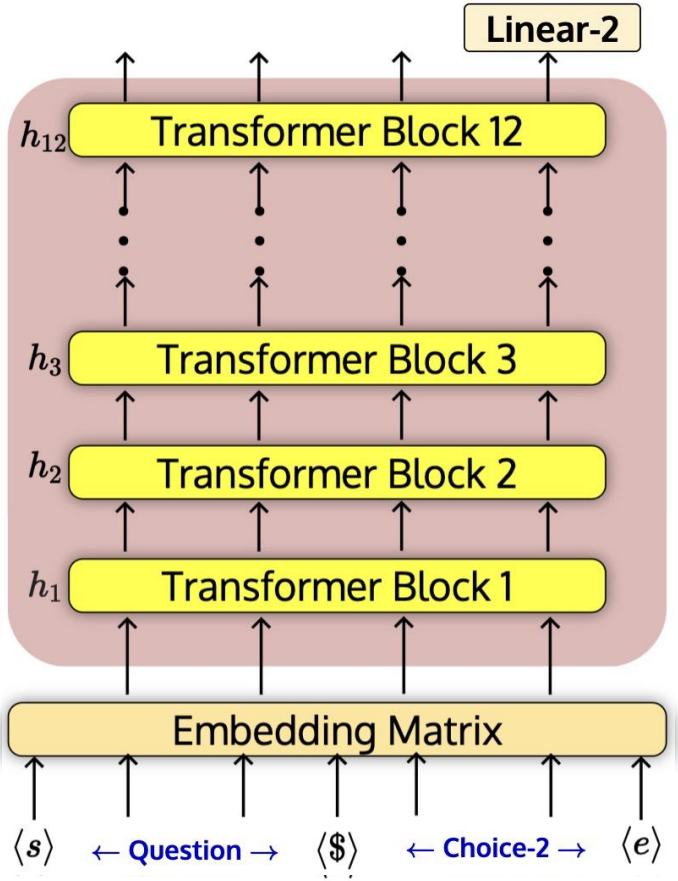
Textual Entailement/Contradiction

Text: A soccer game with multiple males playing
Hypothesis: Some men are playing a sport
Entailment: True

Note: use a delimiter token ⟨$⟩ to differentiate the text from the hypothesis.



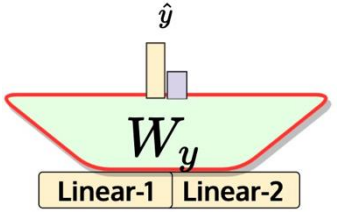
SFT - Example



Multiple Choice

Question: Which of the following animals is an amphibian?
Choice-1: Frog
Choice-2: Fish

Feed in the question along with the choice-n
For all choices, Normalize via softmax



SFT - Example

Text Generation

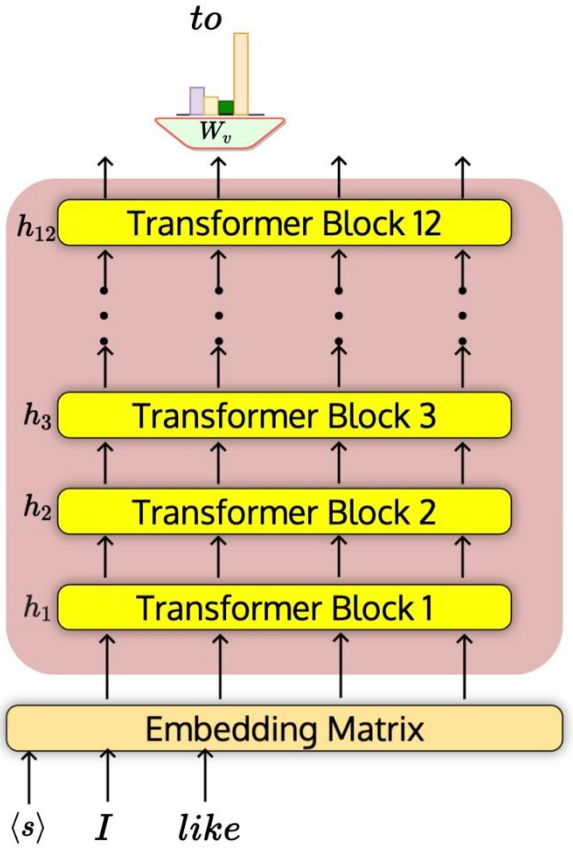
Input:
Prompt: I like

$$M = \begin{bmatrix} 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 & -\infty \end{bmatrix}$$

Feed in the prompt along with the mask and run the model in autregressive mode

Stopping Criteria:
Sequence length: 5
or outputting a token: <e>

Output:
I like to think that



SFT - Examples

Scaling Up Instruction Finetuning



SFT - Results

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:
(A) They will discuss the reporter's favorite dishes
(B) They will discuss the chef's favorite dishes
(C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.
The reporter and the chef will discuss the reporter's favorite dishes.
The reporter and the chef will discuss the chef's favorite dishes.
The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✖ (doesn't answer question)

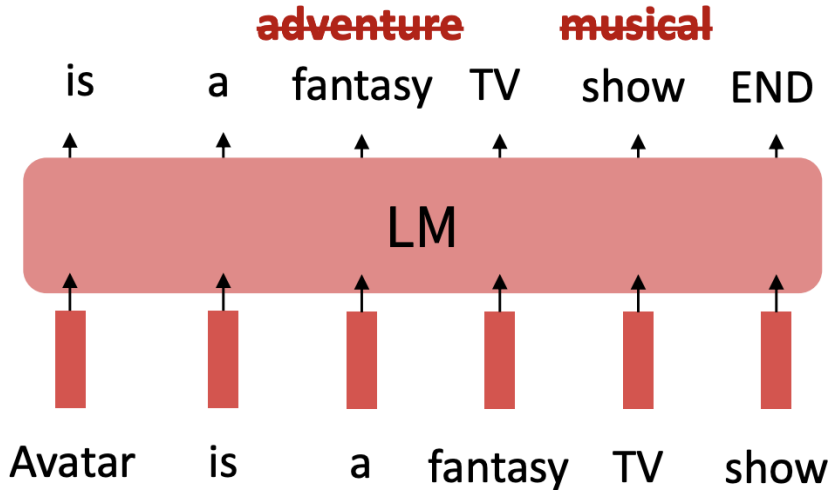
Flan-PaLM 540B output

The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C). ✔

trying FLAN-T5 out to get a sense of its capabilities:
https://huggingface.co/google/flan-t5-xxl

Limitations of SFT

- **Expensive** to collect groundtruth data for so many tasks.
- Tasks like open-ended creative generation **have no standard answers**.
  - *Write a story about a dog and her pet grasshopper.*
- Language modeling **penalizes** all token-level mistakes **equally**, but some errors are worse than others.
- Mismatch between LM objective and human preferences



Can we explicitly attempt to satisfy human preferences?

Common roadmap for LLMs

- Phase 1: pre-training
  - Learn **general** world knowledge, ability, etc.
- Phase 2: Supervised finetuning
  - Tailor to **tasks** (**unlock** some abilities)
- Phase 3: RLHF
  - Tailor to **humans**
  - *Even you could teach ChatGPT to do something*